StoryCast: Project Research & Planning Document

Repository:https://github.com/charleswereangoye/storycast

1. Project Overview & Goals

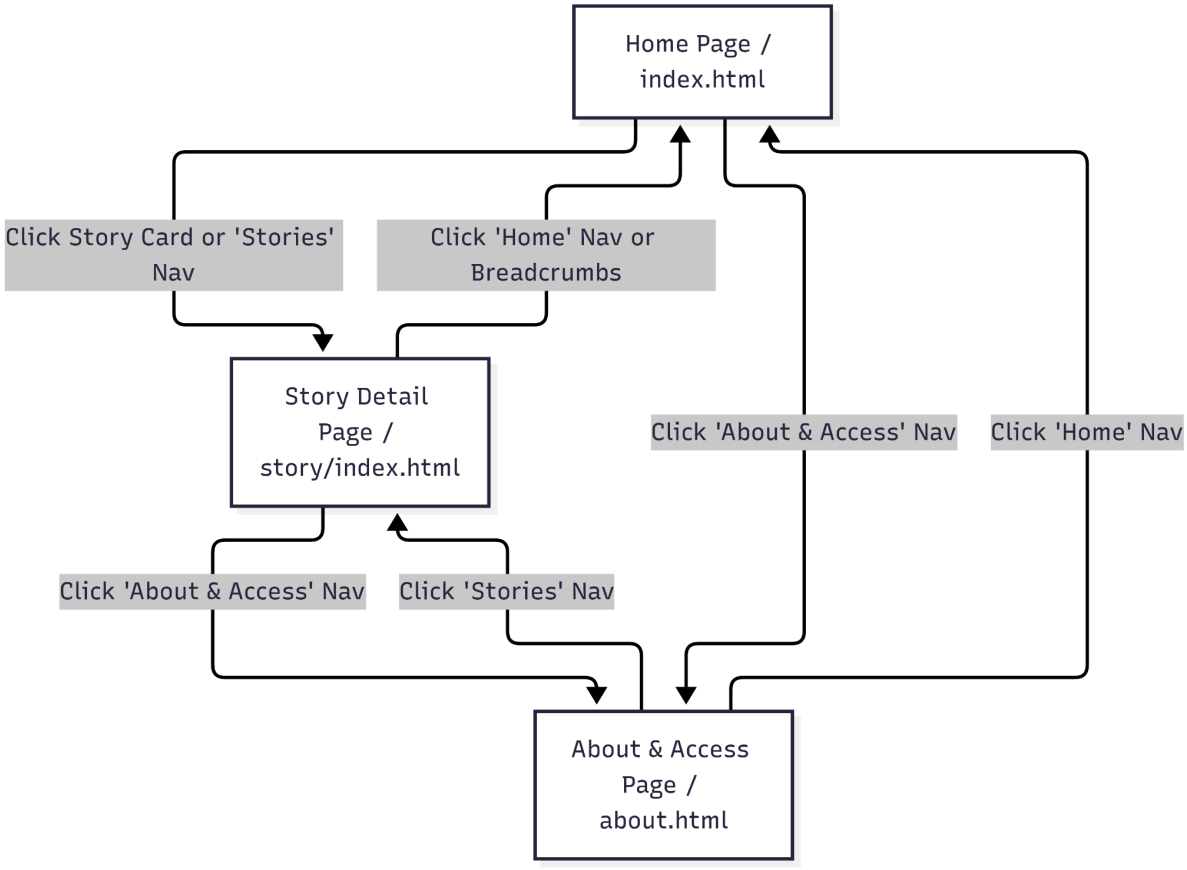
StoryCast is a 3-page media microsite created to showcase audio and video storytelling in a clean, welcoming and accessible environment. My goal for this project was to prove that modern high-quality websites can be built entirely using pure web standards (Semantic HTML5, Sass and native CSS/JavaScript) without relying on heavy third-party frameworks like React or Tailwind.

A central priority of the project is digital accessibility (WCAG 2.1 AA standards). Media websites often create barriers for users with hearing, visual or physical disabilities. I designed StoryCast to address this by making sure every story includes closed captions, complete written transcripts, high contrast ratios and full keyboard-friendly navigation.

Key Objectives:

- **Framework-Free Development:** Build the entire project using native HTML5, modular Sass (SCSS), and lightweight vanilla JavaScript.
- **Inclusive by Design:** Ensure every audio and video asset provides captions and text transcripts so deaf and hard-of-hearing users are never excluded.
- **Keyboard & Screen Reader Friendly:** Ensure someone using only a keyboard or screen reader can navigate every page smoothly.
- **Modern CSS Techniques:** Implement CSS Grid, Flexbox, and CSS Container Queries so components adapt dynamically to their immediate layout container.

2. Information Architecture (IA) & Sitemap



Here is how the 3 pages are structured and connected. Navigation is consistent across all pages through a persistent top header, a mobile-friendly slide menu, breadcrumb navigation on inner pages and a unified footer.

Page Breakdown:

1. **Home Page (index.html)**
   - **Purpose:** Introduces the platform and showcases recent stories.
   - **Key Sections:** Persistent accessible header, hero section highlighting a documentary, accessibility mission callout, a responsive grid of recent story cards and the footer.
2. **Story Detail Page (story/index.html)**
   - **Purpose:** Provides the full, deep-dive media experience for a single story.

Key Sections: Breadcrumb navigation, an accessible video player (using `<figure>` and `<video>` tags with `.vtt` captions), an interactive text transcript reader and a two-card "Related Stories" section.

3. **About & Access Page (about.html)**

Purpose: Explains the StoryCast mission and outlines our specific accessibility standards.

Key Sections: Mission values, a grid explaining the 4 core accessibility pillars (Captions, Transcripts, Keyboard Controls and Contrast) and a direct contact card for accessibility support.

Navigation Flow:

The Home page links directly to the Story Detail page through the story cards.

The Story Detail page connects back to Home via breadcrumbs and the logo, or to the About & Access page.

The About & Access page links back to the home page and stories through the main header navigation.

3. Wireframes & Design Artifacts

The visual design, color tokens and layout measurements were translated directly from my Figma wireframes and high-fidelity prototypes:

Figma Wireframes: View Wireframes

Figma High-Fidelity Mockups: View High-Fidelity Design

4. Technical Architecture Decisions

A. Semantic HTML5 (Avoiding "Div Soup")

Instead of building the layout with endless unlabelled `<div>` tags, I made sure to use standard HTML5 semantic tags to give the document real meaning:

`<header>` and `<nav>` for site-wide navigation.

`<main id="main-content">` to wrap primary page content (targeted by my skip link).

`<article>` for self-contained story cards and transcript sections.

`<figure>` and `<figcaption>` to pair media players with their descriptions.

This ensures screen readers can announce sections properly and let users jump straight to landmarks.

B. Modular Sass (SCSS) Architecture

To keep styles clean and easy to maintain, I split the styles into modular partials using Sass:

- `_colors.scss`: Stores color variables and tokens to ensure strict contrast compliance.
- `_typography.scss`: Defines font scales and line heights.
- `_spacing.scss`: Controls layout gaps and paddings.
- `main.scss`: Compiles all partials and organizes rules to avoid specificity conflicts.

C. Smart Story Cards with CSS Container Queries

Traditional media queries only check the width of the entire browser window. I wanted the story cards to be modular, so I applied a CSS Container Query to the card wrapper.

- When the card container has 640px or more of available space, it spreads out into a horizontal layout.
- When the container shrinks (like on mobile or in a sidebar) it automatically stacks vertically into a compact preview. This makes the cards truly responsive based on their own environment.

5. Accessibility (WCAG 2.1 AA) Implementation

I planned and implemented every accessibility requirement from the assignment rubric:

- **Closed Captions:** Every video and audio player includes a native `<track kind="captions">` pointing to an accessible `.vtt` file.
- **Complete Transcripts:** The Story Detail page includes a dedicated Transcript Section directly beneath the media player so hard-of-hearing users are not excluded.
- **Keyboard-First Navigation:** A hidden "Skip to Content" link is placed at the top of each page. All interactive elements have visible focus rings

(`:focus-visible`) so keyboard navigators can easily see where they are tabbing.

High-Contrast Colors: All color combinations meet or exceed WCAG AA contrast ratios (e.g. dark charcoal text on a white background or yellow accent).

ARIA Attributes: Decorative SVG icons are marked with `aria-hidden="true"` and interactive buttons without text use clear `aria-label` attributes.

6. Student Reflection & Takeaways

Working on StoryCast really changed the way I think about frontend development:

1. **Accessibility is not an afterthought:** Designing with accessibility in mind from the wireframe stage made writing the HTML so much cleaner than trying to "fix" it at the end.
2. **Container Queries are amazing:** They solved the problem of card responsiveness far more cleanly than relying strictly on standard media queries.
3. **The power of native web standards:** Building this without frameworks helped me truly appreciate the power of semantic HTML5 and a clean Sass architecture. The resulting site is lightweight, fast and accessible to everyone.